

DL for ENG

LECTURE 7.2

Fundamental PDEs

Remus Teodorescu

*With support from Research Assistant Noman Khan -
nomank@energy.aau.dk*

Aalborg University · ret@et.aau.dk

Deep Learning for Engineering · 2026

CLASSIFY · STEADY · IN TIME

Poisson	elliptic: a source, every wall
Helmholtz	a mode and its eigenvalue
heat	parabolic: one initial field
wave	hyperbolic: two initial conditions

Every equation of Part 2 is one of four. Poisson and Helmholtz are steady: a condition on every wall and none in time, and Helmholtz asks for a mode and its eigenvalue together. Heat and wave add time as an input, and one or two initial conditions.

Learning Objectives

SECTIONS, IN ORDER

- 1 **Classification** — elliptic, parabolic, hyperbolic
- 2 **Poisson** — the steady reference, and its conditions
- 3 **Helmholtz** — a mode and an eigenvalue, and $u \equiv 0$
- 4 **Heat** — time as an input, and one initial field
- 5 **Wave** — two initial conditions, and sine activations

READ, AND WHAT EACH ANSWERS

- | | |
|-------------|--------------------------------------|
| Liu ch. 2 | which conditions make it well posed? |
| Liu ch. 2.4 | what does a steady balance need? |
| Liu ch. 6 | how is a mode found, not zero? |
| Liu ch. 4.1 | what does time add to Poisson? |
| Liu ch. 5 | why two initial conditions? |

PDE Classification

PDE classification — a second-order linear equation is elliptic, parabolic or hyperbolic according to the sign of its discriminant $B^2 - 4AC$. The type decides which conditions make the problem well posed: boundary values only, one initial condition as well, or two.

ONE DISCRIMINANT, THREE TYPES

	Δ	example	conditions
elliptic	< 0	Poisson	boundary only
parabolic	$= 0$	heat	boundary, one initial
hyperbolic	> 0	wave	boundary, two initial

$A u_{xx} + B u_{xy} + C u_{yy} + D u_x + E u_y + F u = G$

$\Delta = B^2 - 4AC$

THE TYPE IN PYTHON

```
def pde_type(A, B, C):  
    # the coefficients of u_xx, u_xy, u_yy  
    d = B**2 - 4*A*C          # the discriminant: its sign decides  
    return 'elliptic' if d < 0 else 'parabolic' if d == 0 else 'hyperbolic'
```

THE FOUR EQUATIONS OF TODAY

	type	what is new
Poisson	elliptic	the reference
Helmholtz	eigenvalue	λ unknown too
heat	parabolic	time as an input
wave	hyperbolic	two initial fields

Helmholtz sits outside the three types: an eigenvalue problem, with a failure the other three do not have.

Poisson Equation

Poisson equation — the steady balance between diffusion and a distributed source, $u_{xx} + u_{yy} + f = 0$, with no time. It is elliptic: every point feels every boundary at once, so it needs a value or a flux at every boundary point and no initial condition.

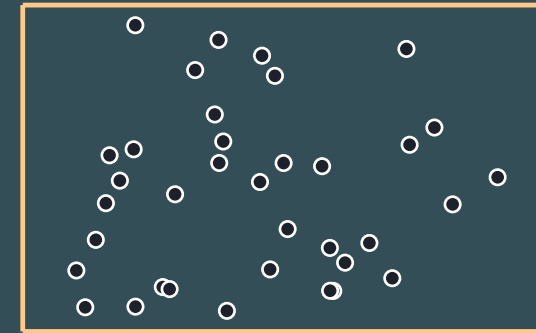
KIND, CONDITIONS, APPLICATIONS

- **kind** — elliptic, no time
- **conditions** — a value or a flux on every wall
- **applications** — conduction, electrostatics

$$u_{xx} + u_{yy} + f(x, y) = 0 \quad u = \bar{u} \text{ on } \partial\Omega_D, \quad \frac{\partial u}{\partial n} = \bar{t} \text{ on } \partial\Omega_N$$

TALK · 2 MIN How would you explain what makes a problem well posed, in your own words? Then, in the figure on the right, what fixes the level of the solution, and what would be lost if the solid walls were Neumann too?

A MIXED PROBLEM



solid: Dirichlet · dashed: Neumann

POISSON IN PYTORCH

```
F = u_xx + u_yy + f          # the residual, inside
B_D = net(x_D) - u_bar        # Dirichlet: the value
B_N = dudn(net, x_N) - g      # Neumann: the flux
```

Helmholtz Equation

Helmholtz equation — Poisson's equation with the source replaced by λu : $\nabla^2 u + \lambda u = 0$. Only certain pairs of a mode u and an eigenvalue λ solve it, so both are unknown together, and a mode is fixed only up to its scale. It arises whenever the wave equation is separated in time.

KIND, CONDITIONS, APPLICATIONS

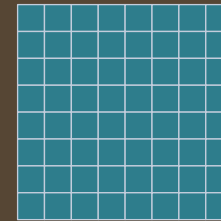
- **kind** — an eigenvalue problem
- **conditions** — the walls, and a fixed norm
- **applications** — vibration and acoustic modes

$$\nabla^2 u + \lambda u = 0, \quad \lambda = k^2 \quad u_{mn} = 2\sin(m\pi x)\sin(n\pi y), \quad \lambda_{mn} = (m^2 + n^2)\pi^2$$

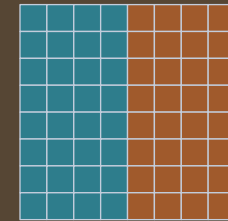
HELMHOLTZ IN PYTORCH

```
lam = nn.Parameter(torch.tensor(15.0)) # λ, unknown, trained beside θ
F = u_xx + u_yy + lam * u             # the residual
loss = F.pow(2).mean() + B.pow(2).mean() # the residual and the walls
```

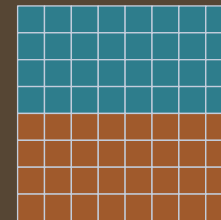
FIRST FOUR MODES OF THE UNIT SQUARE



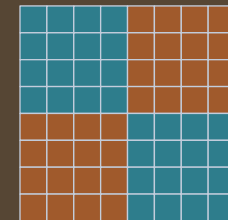
(1,1) $\lambda = 2\pi^2$



(2,1) $\lambda = 5\pi^2$



(1,2) $\lambda = 5\pi^2$



(2,2) $\lambda = 8\pi^2$

Two modes share $\lambda = 5\pi^2$; any mix of them is a mode.

Trivial Solution

Trivial solution — $u \equiv 0$ solves the Helmholtz equation and its walls for any λ , with a residual of exactly zero, and gradient descent finds it first. The loss must exclude it: a normalisation term fixes the scale and a floor keeps λ away from zero.

A PERFECT SCORE FOR A WRONG ANSWER

$$u \equiv 0, \quad \lambda = 0$$

satisfies the equation and the walls exactly

- **the expected outcome** — not a rare failure
- **two safeguards** — a unit norm and a floor on λ

THE SAFEGUARDS IN PYTORCH

```
L_norm = (u.pow(2).mean() - 1).pow(2)      # unit norm: u ≡ 0 fails it
lam = lam0 + nnf.softplus(lam_raw)          # a floor: λ stays above λ₀
loss = L_pde + L dbc + w_n * L_norm          # zero only for a true mode
```

WHAT SATISFIES WHICH TERM

	residual	walls	norm = 1
$u \equiv 0, \lambda = 0$	yes	yes	no
mode (1,1)	yes	yes	yes
mode (2,1)	yes	yes	yes
$2 \times \text{mode (1,1)}$	yes	yes	no

Only the norm column tells the trivial solution from a mode. On the unit square the lowest eigenvalue is $2\pi^2 \approx 19.74$: a λ far from it is a bug.

Orthogonality

Orthogonality — two modes with different eigenvalues are orthogonal: the integral of their product over the domain is zero. Asking each new mode to be orthogonal to those already found, and of unit norm, makes each run find the next mode instead of the lowest again.

ONE MODE AT A TIME

$$\langle u_{mn}, u_{pq} \rangle = \int_{\Omega} u_{mn} u_{pq} d\Omega = \delta_{mp} \delta_{nq}$$

distinct modes are orthogonal, and each has unit norm

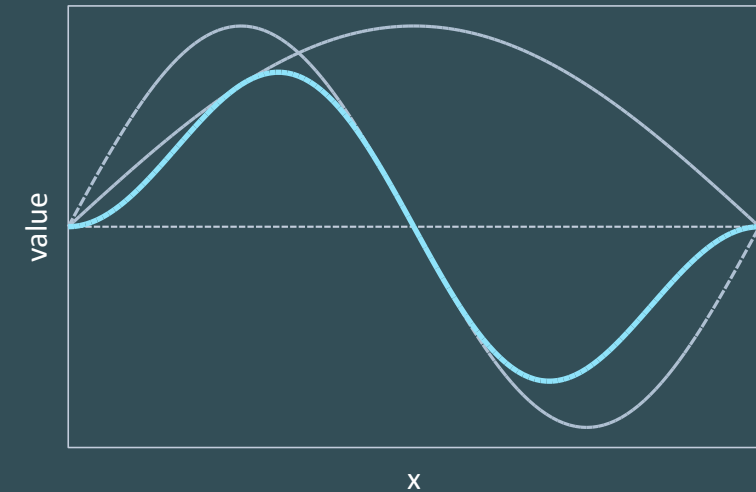
$$\mathcal{L} = \mathcal{L}_{PDE} + w_o \mathcal{L}_{orth} + w_n \mathcal{L}_{norm}, \quad \lambda \geq \lambda_0$$

the residual, plus orthogonality to the modes found, plus the norm

THE NEXT MODE IN PYTORCH

```
L_orth = sum((u * uk).mean().pow(2) for uk in found) # <u, u_k>^2, lower modes
L_norm = (u.pow(2).mean() - 1).pow(2) # unit norm
loss = L_pde + w_o * L_orth + w_n * L_norm # finds the next mode
```

ORTHOGONALITY, VISUALLY



Grey $\sin \pi x$, dashed $\sin 2\pi x$, cyan their product: equal positive and negative area, so its integral is zero.

Heat Equation

Heat equation — the transient version of Poisson's equation, $u_t = \alpha(u_{xx} + u_{yy}) + f$. It is parabolic, first order in time: besides the boundary for all t it needs one initial field, time becomes a third input, and every disturbance spreads and decays.

KIND, CONDITIONS, APPLICATIONS

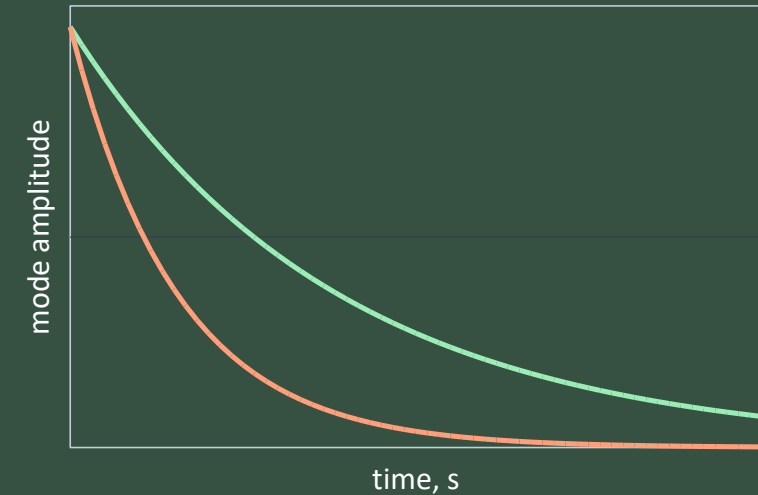
- **kind** — parabolic, first order in time
- **conditions** — the walls, and one initial field
- **applications** — a die after a power pulse

$$u_t = \alpha (u_{xx} + u_{yy}) + f(x, y, t) \quad u(x, y, 0) = f_{IC}(x, y)$$

HEAT IN PYTORCH

```
u = net(torch.cat([x, y, t], 1))          # time is a third input
F = grad(u, t) - alpha * (u_xx + u_yy) - f # the residual; grad = autograd
L_ic = (net(xy0t) - f_ic).pow(2).mean()    # the initial field, soft
u_hard = f_ic + t * Dxy * net(xyt)        # or built in: exact at t = 0
```

THE DIE'S TWO MODES DECAY



Ex_07.2's die: mode (1,1) in green decays at 17.4 /s, mode (2,1) in orange at 43.4 /s, so the field changes shape, not only size.

Wave Equation

Wave equation — $u_{tt} = c^2(u_{xx} + u_{yy}) + f$: a disturbance travels at the finite speed c and keeps its shape instead of decaying. It is hyperbolic, second order in time, so it needs the boundary and two initial conditions. The panel struck in Ex_07.2 rings at 141 Hz.

KIND, CONDITIONS, APPLICATIONS

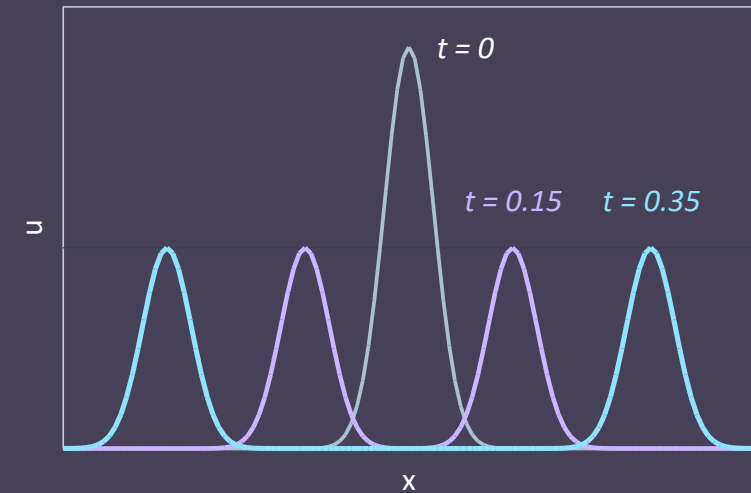
- **kind** — hyperbolic, second order in time
- **conditions** — the walls, a shape and a velocity
- **applications** — a struck panel, acoustics

$$u_{tt} = c^2 (u_{xx} + u_{yy}) + f(x, y, t) \quad u(x, y, 0) = u_0(x, y), \quad u_t(x, y, 0) = v_0(x, y)$$

THE WAVE IN PYTORCH

```
u_tt = grad(grad(u, t), t)           # second order in time
F = u_tt - c**2 * (u_xx + u_yy) - f  # the residual
L_ic = (net(xy0t) - u_0).pow(2).mean() # the shape at t = 0
```

A PULSE SPLITS AND TRAVELS



d'Alembert's solution: the pulse splits into two halves that travel at c and keep their shape. Nothing decays.

Two Initial Conditions

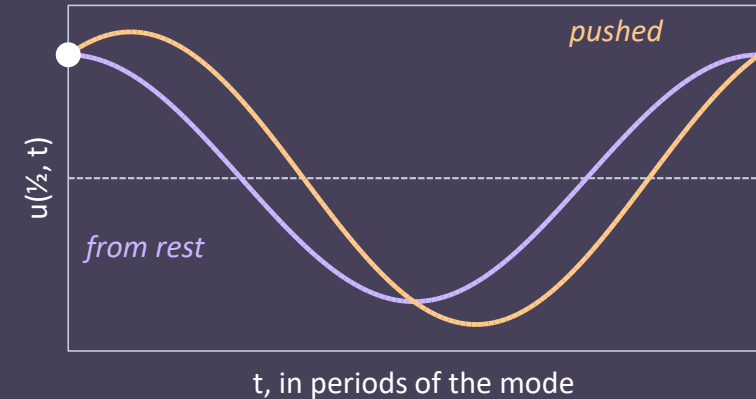
Initial velocity — the second initial condition of an equation that is second order in time: the rate u_t at $t = 0$. The same starting shape released from rest or given a push has two different futures, so without it the problem is under-determined.

THE SAME SHAPE, TWO FUTURES

- **the rule** — as many initial conditions as time derivatives
- **left out** — $u \equiv 0$ satisfies everything else
- **the loss** — converges, and the panel never moves

TALK • 2 MIN How would you explain why a second-order equation needs two initial conditions, in your own words? Then, in the plot on the right, what would a network trained without the velocity condition produce, and why would its loss look good?

u AT THE CENTRE, AGAINST TIME



THE VELOCITY IN PYTORCH

```
L_iv = (u_t0 - v_0).pow(2).mean() # velocity at t = 0
loss = L_pde + L dbc + L_ic + L_iv # drop L_iv: u ≡ 0
```

Periodic Activations

Periodic activations — a first layer that uses $\sin(\omega_0 z)$ instead of tanh. A wave is a sum of sinusoids, and the derivative of a sinusoid is again a sinusoid, so the network can hold oscillations that a tanh network smooths into their envelope.

MATCH THE NETWORK TO THE SOLUTION

$$\varphi(\mathbf{x}) = \sin(\omega_0 \mathbf{W}\mathbf{x} + \mathbf{b}), \quad \omega_0 \approx 30$$

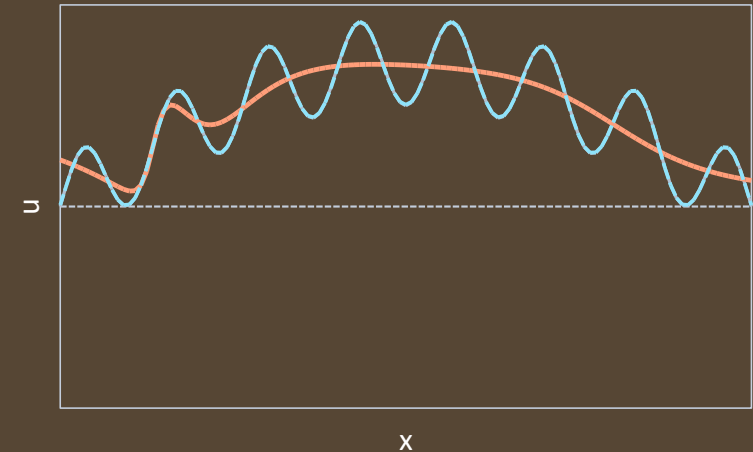
ω_0 sets the frequencies the first layer reaches at the start

- **oscillations** — kept, not smoothed away
- **a fair test** — the same size, the same steps

A SINE LAYER IN PYTORCH

```
class Sine(nn.Module):
    # the periodic activation
    def forward(self, z): return torch.sin(30 * z) #  $\omega_0 = 30$ 
net = nn.Sequential(nn.Linear(3, 64), Sine(), nn.Linear(64, 1)) # sine layer
```

MEASURED · TANH AGAINST SINE



tanh 32 x 3 2209 weights, error 25.7 %

sine 32 x 3 2209 weights, error 0.53 %

Orange tanh, dashed cyan sine, grey the target. 3000 Adam steps, lr 1e-3, seed 0.

Advanced Topics

RELATED TOPICS LEFT OUT, AND WHERE THEY MATTER

topic	what it adds	read	where it makes a difference
learned boundary	hard walls without a polynomial	Liu ch. 3.1 – 3.3	a plate with a hole
space–time training	the whole time block at once	Liu ch. 4.2 – 4.3	long transients
causal training	early times fitted first	Liu ch. 4.4	fast transients
many modes	K modes, each orthogonal	Liu ch. 6.5	vibration spectra

WHY THEY ARE LEFT OUT

Each returns with a device: the plate with a hole and the transient die in L8, and Burgers' equation and its shock with the flow of L9. None is needed to solve today's four equations.

Exercise 7.2

THE NOTEBOOKS, IN ORDER

- **00** — checks the space–time sample and u_t
- **01** — the die, with a soft initial field
- **02** — the die, with the initial field built in
- **03** — the struck panel, both conditions
- **04** — the panel without its velocity condition
- **05** — the report; mini projects MP7.2A – C
- **optional** — Helmholtz in a stator slot, on its own

THE QUESTION EACH NOTEBOOK ANSWERS

- 01** where in time is the error?
- 02** what does a built-in start buy?
- 03** does the panel keep its period?
- 04** why is the lower loss wrong?

REPORT QUESTION, TO CONCLUDE

The die's error fell with time and the panel's grew, and the panel without its velocity condition reached the lowest loss of all. What does the type of an equation tell you before training, and what can a low loss not tell you?

Key Takeaways

STEADY

what it is	elliptic: every wall felt at once
function	Poisson a source; Helmholtz a mode
train, tune	a unit norm, and a floor on λ
vs in time	no initial condition, no time input
used for	conduction, potentials, modes

IN TIME

what it is	heat parabolic, wave hyperbolic
function	diffusion decays; a wave travels
train, tune	the initial field; sine for waves
vs steady	time an input; one or two ICs
used for	a die after a pulse, a panel

Next — L8.1: steady heat on real geometry, a plate with a cooling hole.

Questions

1 – 5 · STEADY

- 1 What decides the type of a second-order equation, and which conditions does each type need? *Liu ch. 2; Ex_07.2 notebook 04*
- 2 What does Poisson's equation describe, why does it need a condition on every wall, and what goes wrong with a flux on all of them? *Liu ch. 2.4; Ex_07.2 notebook 04*
- 3 How is a Helmholtz network trained: why does it find $u \equiv 0$, and which two safeguards exclude it? *Liu ch. 6.2, 6.4.3; Ex_07.2 supplement*
- 4 How does a steady problem differ from one in time: which input and which conditions does it do without? *Liu ch. 2, 4.1*
- 5 What are Poisson and Helmholtz used for, and how does orthogonality find the modes after the first? *Liu ch. 6.4 – 6.5*

6 – 10 · IN TIME

- 6 What does the heat equation add to Poisson, and what does its type do to an error made early? *Liu ch. 4.1; Ex_07.2 notebook 03*
- 7 How is the heat equation's initial field imposed, soft or built in, and why is it the term a PINN most often loses? *Liu ch. 4.1 – 4.2; Ex_07.2 notebooks 01 – 02*
- 8 How is a wave network tuned: what does a sine activation buy, and how must it be compared with tanh? *Liu ch. 5.7.4; Ex_07.2 notebook 03*
- 9 How does the wave equation differ from the heat equation, and why does it need two initial conditions? *Liu ch. 5.1 – 5.2; Ex_07.2 notebook 03*
- 10 What are heat and wave used for, and what did the panel without its velocity condition show about a low loss? *Liu ch. 5.1 – 5.2; Ex_07.2 notebook 04*

NEXT

L8.1 — Stationary Heat

Poisson's equation with real units, a convection wall and a plate with a cooling hole.

BEFORE THE NEXT SESSION

read	Liu ch. 4.1, 5.1 – 5.2, 6.1
run	Ex_07.2 notebooks 00 to 04
bring	the panel's loss curves, both runs
think about	which type your own problem is

Block L7 is finished: the method, and the four equations it is applied to.